

Assignment 3: Persistence and Data Management

Course: CS6650

Project: ChatFlow

Repository: <https://github.com/yimdx/ChatFlow>

Date: 2026-04-06

1. Overview

Assignment 3 extends the Assignment 2 queue-based architecture by adding persistent storage, query APIs, and analytics for chat activity. The design goal is to preserve the real-time behavior of ChatFlow while handling high write volume without turning the database into a bottleneck.

This document provides: - A concrete database design for required core and analytics queries. - A **consumer-v3** write-behind architecture with batching and fault handling. - Metrics and query APIs hosted in **server-v2**. - Load-test plan, acceptance targets, and reporting format.

2. Implementation Scope

The repository already includes: - **server-v2/** with RabbitMQ producer flow. - **consumer/** with multithreaded queue consumption and metrics endpoint. - **client-part2/** with per-message CSV metrics and percentile analysis. - **monitoring/** scripts for queue and system-level metrics.

Assignment 3 adds: - Persistent message storage. - Query/analytics APIs backed by DB. - Database-focused performance tuning and resilience logic.

3. Database Choice and Rationale

3.1 Selected Database

PostgreSQL 15+ (single writer node to start, read replica optional later).

3.2 Why PostgreSQL

- Strong consistency and transactional guarantees for message durability.
- Excellent support for ordered range queries with composite indexes.
- Mature tooling for partitioning, backups, and observability.
- Supports `INSERT ... ON CONFLICT` for idempotent writes.
- Straightforward SQL for analytics and materialized views.

3.3 Tradeoffs

- Write-heavy workloads need careful batching and index discipline.
- Distinct-count analytics can become expensive without pre-aggregation.
- Partitioning strategy is required for long-term retention performance.

4. Data Model

4.1 Logical Model

Primary entity is `chat_messages` (append-heavy). Secondary table `user_room_activity` supports fast participation queries. Rollup table/materialized view supports analytics reads.

4.2 Schema (SQL)

```
CREATE TABLE IF NOT EXISTS chat_messages (  
  message_id      UUID PRIMARY KEY,  
  room_id         INT NOT NULL,  
  user_id         BIGINT NOT NULL,  
  username        VARCHAR(64) NOT NULL,  
  message_type    VARCHAR(16) NOT NULL,  
  message_text    TEXT NOT NULL,  
  message_ts      TIMESTAMPTZ NOT NULL,  
  server_id       VARCHAR(64) NOT NULL,  
  client_ip       INET,  
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE IF NOT EXISTS user_room_activity (  
  user_id         BIGINT NOT NULL,  
  room_id         INT NOT NULL,  
  first_seen_ts   TIMESTAMPTZ NOT NULL,  
  last_seen_ts    TIMESTAMPTZ NOT NULL,  
  message_count   BIGINT NOT NULL DEFAULT 0,  
  PRIMARY KEY (user_id, room_id)  
);
```

```
CREATE TABLE IF NOT EXISTS message_minute_stats (  
  bucket_minute   TIMESTAMPTZ NOT NULL,  
  room_id         INT NOT NULL,  
  user_id         BIGINT NOT NULL,  
  msg_count       BIGINT NOT NULL,  
  PRIMARY KEY (bucket_minute, room_id, user_id)  
);
```

4.3 Indexing Strategy

-- Core query 1: room + time range

```
CREATE INDEX IF NOT EXISTS idx_messages_room_ts  
  ON chat_messages (room_id, message_ts);
```

-- Core query 2: user history + optional date range

```

CREATE INDEX IF NOT EXISTS idx_messages_user_ts
  ON chat_messages (user_id, message_ts DESC);

-- Core query 3: active users in time range (distinct user_id)
CREATE INDEX IF NOT EXISTS idx_messages_ts_user
  ON chat_messages (message_ts, user_id);

-- Core query 4: participated rooms by user
CREATE INDEX IF NOT EXISTS idx_activity_user_last
  ON user_room_activity (user_id, last_seen_ts DESC);

```

Notes: - Keep indexes minimal to protect write throughput. - Use monthly partitioning on chat_messages.message_ts once volume grows.

5. Required Query Coverage

5.1 Core Query 1: Messages in Room + Time Range

```

SELECT message_id, room_id, user_id, username, message_type, message_text, message_ts
FROM chat_messages
WHERE room_id = $1
  AND message_ts >= $2
  AND message_ts <= $3
ORDER BY message_ts ASC
LIMIT $4 OFFSET $5;

```

Target: <100ms for 1000 rows.

5.2 Core Query 2: User Message History

```

SELECT message_id, room_id, message_type, message_text, message_ts
FROM chat_messages
WHERE user_id = $1
  AND ($2::timestampz IS NULL OR message_ts >= $2)
  AND ($3::timestampz IS NULL OR message_ts <= $3)
ORDER BY message_ts DESC
LIMIT $4 OFFSET $5;

```

Target: <200ms.

5.3 Core Query 3: Active Users in Window

```

SELECT COUNT(DISTINCT user_id) AS active_users
FROM chat_messages
WHERE message_ts >= $1
  AND message_ts <= $2;

```

Target: <500ms.

5.4 Core Query 4: Rooms a User Participated In

```
SELECT room_id, last_seen_ts, message_count
FROM user_room_activity
WHERE user_id = $1
ORDER BY last_seen_ts DESC;
```

Target: <50ms.

6. Analytics Query Coverage

6.1 Messages per Second / Minute

Use `date_trunc('second'|'minute', message_ts)` on `chat_messages` for ad hoc analysis, or query `message_minute_stats` for production dashboards.

6.2 Most Active Users

```
SELECT user_id, COUNT(*) AS total_messages
FROM chat_messages
WHERE message_ts BETWEEN $1 AND $2
GROUP BY user_id
ORDER BY total_messages DESC
LIMIT $3;
```

6.3 Most Active Rooms

```
SELECT room_id, COUNT(*) AS total_messages
FROM chat_messages
WHERE message_ts BETWEEN $1 AND $2
GROUP BY room_id
ORDER BY total_messages DESC
LIMIT $3;
```

6.4 Participation Patterns

```
SELECT user_id,
       COUNT(DISTINCT room_id) AS rooms_joined,
       MIN(message_ts) AS first_activity,
       MAX(message_ts) AS last_activity,
       COUNT(*) AS total_messages
FROM chat_messages
WHERE message_ts BETWEEN $1 AND $2
GROUP BY user_id
ORDER BY total_messages DESC
LIMIT $3;
```

7. Metrics API Contract

Add endpoints to **server-v2** (it hosts the DB-backed retrieval API):

- GET /api/v1/messages/room/{roomId}?start=...&end=...&limit=...&offset=...
- GET /api/v1/messages/user/{userId}?start=...&end=...&limit=...&offset=...
- GET /api/v1/analytics/active-users?start=...&end=...
- GET /api/v1/analytics/user-rooms/{userId}
- GET /api/v1/analytics/top-users?start=...&end=...&n=...
- GET /api/v1/analytics/top-rooms?start=...&end=...&n=...
- GET /api/v1/analytics/messages-rate?start=...&end=...&granularity=second|minute

Example aggregate response format:

```
{
  "window": {
    "start": "2026-03-27T00:00:00Z",
    "end": "2026-03-27T00:30:00Z"
  },
  "core": {
    "activeUsers": 1241,
    "userRooms": [{"roomId": 3, "lastActivity": "2026-03-27T00:29:50Z"}]
  },
  "analytics": {
    "topUsers": [{"userId": 9021, "count": 320}],
    "topRooms": [{"roomId": 12, "count": 18420}],
    "rate": [{"bucket": "2026-03-27T00:29:00Z", "count": 2050}]
  }
}
```

Client action after load test: - Call metrics endpoints automatically. - Save JSON snapshots under **load-tests/results/**. - Print summary in client logs and include screenshot in report.

Expected client behavior: - After the benchmark finishes, call **server-v2** metrics API. - Log the returned JSON in the client console. - Capture a screenshot of that console output and place it in the report.

8. Consumer-v3 Write-Behind Design

8.1 Pipeline

RabbitMQ consumer threads -> in-memory write queue -> DB writer workers -> Postgres

8.2 Thread Pools

- Queue consumers: pull from RabbitMQ and deserialize.
- DB writer workers: batch INSERT ... ON CONFLICT DO NOTHING.

- Stats workers: update minute rollups / activity table asynchronously.

8.3 Batching Policy

Test matrix (up to 5 runs):

Run	Size	Flush In- Batcher- val	Overall Through- put	Main- Phase Through- put	Mean	p50	p95	p99	Max	Success	Failures	Persisted Msgs	Queue State	Notes
B1	100	100ms	1097.03 msg/s	146.48 msg/s	882	1 ms	37 ms	61 ms	129 ms	13,755	5,245	19,982	drained to 0	stable
B2	500	100ms	1217.66 msg/s	55.42 msg/s	260	0 ms	23 ms	146 ms	813 ms	12,427	7,573	19,394	drained to 0	best raw through- put, worse tail la- tency
B3	1000	500ms	1165.03 msg/s	55.76 msg/s	664	1 ms	35 ms	58 ms	131 ms	13,613	3,387	19,965	drained to 0	best over- all bal- ance
B4	5000	500ms	1050.93 msg/s	44.83 msg/s	339	0 ms	37 ms	97 ms	978 ms	12,526	7,474	19,916	drained to 0	slower, worse max la- tency
B5	5000	1000ms	1130.07 msg/s	33.72 msg/s	215	0 ms	29 ms	154 ms	571 ms	11,466	8,534	19,505	drained to 0	stats- writer dead- lock warn- ing ob- served

Observed trend: - Smaller batches reduced worst-case write stalls but did not maximize throughput. - **batch=500** / **flush=100ms** produced the highest raw throughput, but tail latency degraded sharply. - **batch=5000** did not improve throughput enough to justify the larger p99/max latency and reduced stability.

- `batch=1000 / flush=500ms` provided the best throughput/latency/stability trade-off.

Selected default for baseline, stress, and endurance runs: **batch 1000, flush 500ms**.

8.4 Idempotency

- `message_id` is unique primary key.
- DB write uses `ON CONFLICT (message_id) DO NOTHING`.
- Duplicate deliveries from at-least-once queue semantics are safe.

8.5 Error Recovery

- Retry with exponential backoff on transient DB failures.
- Dead-letter queue for poison/invalid records.
- Circuit breaker opens when DB failure rate crosses threshold.
- During open circuit: stop aggressive DB writes, keep bounded buffer, shed load if needed.

9. Performance Optimizations

9.1 Database Tuning

- Prepared statements for all write/read paths.
- Connection pool (HikariCP): start `maxPoolSize=30`, tune per host size.
- Keep transactions short and batch-oriented.
- Materialized view or rollup table for expensive analytics.

9.2 Query Caching

- Cache short-lived analytics (`top-users`, `top-rooms`, `rate`) for 5-15 seconds.
- Do not cache room timeline reads with strict ordering unless window is immutable.

9.3 System Resilience

To satisfy the resilience requirement, this design explicitly evaluates all three options below.

Option 1: Circuit Breaker Pattern

- Purpose: protect PostgreSQL when transient failures or saturation begin.
- Trigger condition: rolling DB write failure rate exceeds threshold (example: >20% over last N batches).
- Open behavior: pause aggressive writes, keep bounded queueing, route repeated failures to retry/DLQ path.

- Half-open behavior: allow a small probe window; close circuit only after sustained success.
- Benefit: prevents retry storms and protects downstream database recovery.

Option 2: Rate Limiting

- Purpose: smooth traffic spikes from RabbitMQ ingest into DB writer workers.
- Strategy: token-bucket or semaphore-based in-flight limits per writer worker.
- Enforcement point: before batch flush/persist path and optionally before server broadcast calls.
- Overflow policy: short wait, then nack/requeue or drop-to-DLQ based on reliability policy.
- Benefit: avoids queue-to-DB stampede and stabilizes p95/p99 latency.

Option 3: Caching Layer (Optional)

- Scope: read/query APIs only (hosted in `server-v2`).
- Candidate endpoints: `top-users`, `top-rooms`, `messages-rate`, aggregated `/metrics`.
- TTL recommendation: 5-15s for analytics windows.
- Do not cache: strict ordered room timeline queries for active mutable ranges.
- Benefit: reduces repeated analytics query load on PostgreSQL under dashboard polling.

Selected Combination for This Assignment

Option 1 + Option 2 enabled by default; Option 3 optional.

- Circuit breaker is required for failure isolation.
- Light rate limiting is required for burst control.
- Caching remains optional and can be enabled for analytics-heavy read traffic.

10. Load Testing Results

10.1 Test Definitions

1. Baseline: 500,000 messages, run to completion.
2. Stress: 1,000,000 messages, identify bottlenecks.
3. Endurance: 30 minutes at 80% of max sustainable write rate.

10.2 Metrics to Collect

- Application: ingest throughput, write throughput, p50/p95/p99 write latency.

- Database: CPU, memory, active connections, lock waits, disk IO, buffer hit ratio.
- Queue: depth over time, consumer lag, retry counts, DLQ counts.

10.3 Results Summary

Test	Throughput (msg/s)	p50 (ms)	p95 (ms)	p99 (ms)
Baseline 500k	1778.68	0	7	129
Stress 1M	2110.14	0	11	67
Endurance 30m	875.90 avg	4	48.88	55.51

10.4 Batch Size Optimization Decision

The best-performing configuration for continued testing is **batch size 1000 with a 500ms flush interval**.

Justification: - B2 (500 / 100ms) had the highest raw throughput, but p99 latency (146 ms) and max latency (813 ms) were significantly worse than B3. - B3 (1000 / 500ms) preserved near-top throughput while keeping tail latency low (p99 = 58 ms, max = 131 ms). - B4 and B5 showed that pushing batch size to 5000 did not improve throughput and increased worst-case latency. - Queue snapshots for B2, B3, B4, and B5 all drained to zero by the end of the run, so the choice is driven mainly by throughput/latency balance rather than queue backlog alone.

Because the assignment emphasizes both throughput and real-time behavior, B3 is the most defensible operating point.

10.5 Baseline Test Result (500,000 Messages)

Baseline configuration: - Consumer config: `consumer.batch-1000.flush-500.env`
 - Client load: `TOTAL_MESSAGES=500000, MAIN_PHASE_THREADS=32, WARMUP_THREADS=8, WARMUP_MESSAGES_PER_THREAD=500`

Measured client-side results from `load-tests/results/baseline-500k/client-output.txt`:
 - Successful messages: 161,747 - Failed messages: 338,222 - Total runtime: 281.108 s - Overall throughput: 1778.68 msg/s - Main-phase throughput: 1809.12 msg/s - Mean response time: 6.25 ms - p50: 0 ms - p95: 7 ms - p99: 129 ms - Max response time: 1000 ms

Observed system behavior: - The RabbitMQ overview screenshot (`load-tests/results/baseline-500k/baseline-rabbitmq-overview.png`) shows short queue spikes with a peak of roughly ~110 queued messages, but the queue drained back to zero by the end of the run. - The CloudWatch screenshot (`load-tests/results/baseline-500k/baseline-cloudwatch-summary.png`) shows the messaging tier as the hottest component during the run, with overall

CPU and network activity rising sharply near the end of the test. PostgreSQL did not appear to be the dominant bottleneck in this baseline run. - The saved metrics API response reports 340,188 persisted messages in the measured test window and 96,612 active users, with room 4 as the most active room (34,778 messages).

Interpretation: - This baseline run completed and demonstrated sustained write throughput near 1.8k msg/s at the client level. - Tail latency remained acceptable through p95, but p99 increased to 129 ms, indicating burstiness under heavier main-phase load. - The mismatch between client-reported successes (161,747) and persisted messages recorded by the metrics API (340,188) suggests the primary bottleneck was in the client/server request path and timeout behavior rather than the persistence layer itself. Messages were still being accepted and persisted asynchronously even when the client recorded some sends as failed or timed out. - RabbitMQ queue behavior remained healthy for this load level because backlog spikes were modest and drained quickly instead of growing without bound.

Artifacts saved for the baseline run: - load-tests/results/baseline-500k/client-output.txt
- load-tests/results/baseline-500k/client-part2.log - load-tests/results/baseline-500k/metrics
- load-tests/results/baseline-500k/consumer-tail.txt - load-tests/results/baseline-500k/rabbitmq-overview.png
- load-tests/results/baseline-500k/baseline-rabbitmq-overview.png -
load-tests/results/baseline-500k/baseline-cloudwatch-summary.png

10.6 Stress Test Result (1,000,000 Messages)

Stress configuration: - Consumer config: consumer.batch-1000.flush-500.env
- Client load: TOTAL_MESSAGES=1000000, MAIN_PHASE_THREADS=32, WARMUP_THREADS=8, WARMUP_MESSAGES_PER_THREAD=500

Measured client-side results from load-tests/results/stress-1000k/client-output.txt:
- Successful messages: 325,998 - Failed messages: 658,242 - Total runtime: 473.903 s - Overall throughput: 2110.14 msg/s - Main-phase throughput: 2139.37 msg/s - Mean response time: 6.62 ms - p50: 0 ms - p95: 11 ms - p99: 67 ms - Max response time: 1190 ms

Observed system behavior: - The RabbitMQ overview screenshot (load-tests/results/stress-1000k/stress-rabbitmq-overview.png) shows only brief queue spikes, with total queued messages peaking at roughly ~6 before draining back to zero. This indicates that RabbitMQ remained stable under the 1M-message stress run. - The CloudWatch dashboard screenshot (load-tests/results/stress-1000k/stress-cloudwatch-summary.png) shows the messaging tier and at least one application instance with noticeably higher CPU and network activity during the stress window. The system stayed active across all selected instances, but the workload concentrated most heavily on RabbitMQ and the front-end application tier. - The metrics API response reports 623,450 persisted messages in the captured test window, 99,779 active users, and room 8 as the most active room (49,818 messages). - The

consumer log shows successful, continuous broadcast activity throughout the tail of the run, and the saved consumer tail does not show DB write failures or DLQ growth. - The server log shows successful publish/broadcast activity but also some `WebsocketNotConnectedException` errors near the end of the run, indicating client disconnects or dropped WebSocket sessions under load.

Interpretation: - The stress test achieved higher throughput than the baseline run, increasing from 1778.68 msg/s to 2110.14 msg/s. - Tail latency improved compared with the baseline (p99 = 67 ms vs. 129 ms), suggesting the chosen 1000 / 500ms persistence configuration remained stable at this higher offered load. - The primary degradation signal was not RabbitMQ backlog or DB failure. Instead, the system showed client/server path instability, reflected by a high number of client-reported failures and WebSocket disconnect exceptions on the server. - In other words, the persistence layer continued to absorb and store messages effectively, while the interactive connection layer became the more visible bottleneck during the 1M-message run.

Artifacts saved for the stress run: - `load-tests/results/stress-1000k/client-output.txt`
- `load-tests/results/stress-1000k/client-part2.log` - `load-tests/results/stress-1000k/metrics-a`
- `load-tests/results/stress-1000k/consumer-tail.txt` - `load-tests/results/stress-1000k/server-t`
- `load-tests/results/stress-1000k/server-metrics.json` - `load-tests/results/stress-1000k/rabbit`
- `load-tests/results/stress-1000k/stress-rabbitmq-overview.png` -
`load-tests/results/stress-1000k/stress-cloudwatch-summary.png`

10.7 Endurance Test Result (30 Minutes)

Final endurance configuration: - Consumer config: `consumer.batch-1000.flush-500.env`
- Client load per iteration: `TOTAL_MESSAGES=200000`, `MAIN_PHASE_THREADS=12`,
`WARMUP_THREADS=4`, `WARMUP_MESSAGES_PER_THREAD=200` - Execution pattern:
repeated medium-size benchmark runs for approximately 30 minutes

Important tuning note: - An earlier endurance attempt with a more aggressive client load produced a RabbitMQ backlog of roughly ~168k ready messages. That was too high for a stability-focused endurance test, so the final endurance run was performed with a more modest configuration.

Completed endurance run summary from `load-tests/results/endurance-30m/iter01` through `iter74`: - Iterations completed: 74 - Total successful messages: 1,480,000 - Total failed messages: 27 - Average overall throughput: 875.90 msg/s - Average main-phase throughput: 1110.35 msg/s - Average p50: 4 ms - Average p95: 48.88 ms - Average p99: 55.51 ms - Average max response time: 131.04 ms - Throughput range across iterations: 769.11 msg/s to 935.41 msg/s - First iteration throughput: 869.72 msg/s - Last iteration throughput: 909.75 msg/s - Aggregate benchmark runtime across client iterations: 1692.85 s (plus inter-run delay, giving an overall wall-clock duration of about 30 minutes)

Observed system behavior: - The CloudWatch screenshot (`load-tests/results/endurance-30m/endurance-c`) shows moderate sustained CPU and network usage across the five in-

stances over the endurance window. The server and RabbitMQ tiers were the busiest components, but utilization remained bounded instead of ramping upward continuously. - The RabbitMQ screenshot (`load-tests/results/endurance-30m/endurance-rabbitmq-overview.png`) shows that an overly aggressive attempt temporarily drove queue depth close to 160k, but backlog returned to zero after the load was reduced. This reinforced the decision to use a modest endurance configuration. - Sample metrics API outputs from later endurance iterations reported stable persisted counts of 20,000 messages per iteration and active-user counts around 18k, showing that the persistence pipeline remained healthy throughout the sustained run. - Across the 74 saved client iterations, throughput stayed stable from the first run to the last, and failures remained extremely low (27 out of 1.48M sends).

Interpretation: - The final endurance test demonstrates that ChatFlow can sustain a moderate continuous workload for roughly 30 minutes without unbounded queue growth, widespread client failures, or visible performance collapse. - The most important signal is stability over time: throughput at the end of the run was slightly better than at the beginning, and p95/p99 latency remained tightly clustered instead of drifting upward. - The endurance run also helped identify an operational limit: if client concurrency is pushed too high, RabbitMQ backlog can spike dramatically. However, after reducing the offered load to a sustainable level, the system remained stable and the queue drained normally. - This makes the chosen endurance configuration a reasonable estimate of the system's sustainable operating point on the current single-server deployment.

Artifacts saved for the endurance run: - `load-tests/results/endurance-30m/iter01` through `load-tests/results/endurance-30m/iter74` - `load-tests/results/endurance-30m/endurance-c`
- `load-tests/results/endurance-30m/endurance-rabbitmq-overview.png`

11. Repository Additions Included

The repository now includes:

- `database/`
 - `schema.sql`
 - `indexes.sql`
 - `seed.sql` (optional)
- `consumer-v3/`
 - Updated consumer with write-behind persistence
- `monitoring/`
 - Add DB-focused metrics scripts if not already present
- `load-tests/`
 - Test configs and final result artifacts

12. Configuration Snapshot

- Database URL and credentials source (env vars, no secrets committed).

- Connection pool: min/max, idle timeout, max lifetime.
- Batch config: size, flush interval, retry attempts, backoff base.
- Circuit breaker: failure threshold, open duration, half-open probes.
- Queue config: prefetch, consumer concurrency, DLQ binding.

13. Submission Checklist

- ☒ Database choice and schema documented.
- ☒ Core and analytics query plans documented.
- ☒ Metrics API contract defined.
- ☒ Consumer-v3 write-behind design specified.
- ☒ Idempotency and recovery strategy specified.
- ☒ Load-test scenarios and metric plan documented.
- ☒ Implement **database/** SQL files.
- ☒ Implement **consumer-v3/** persistence code.
- ☒ Execute batch-size optimization tests and fill results table with real numbers.
- ☒ Execute baseline, stress, and endurance tests.
- ☒ Attach metrics API log screenshot and monitoring screenshots.

14. Final Status

Final status: - The end-to-end system is running across AWS EC2 with RabbitMQ, PostgreSQL, **server-v2**, **consumer-v3**, and **client-part2**. - The persistence pipeline, metrics API, idempotent writes, DLQ handling, exponential backoff, and circuit breaker logic are implemented. - The five batch/flush tuning experiments have been executed and analyzed. - The baseline (500,000 messages) experiment has been executed and added to this report. - The stress (1,000,000 messages) experiment has also been executed and added to this report. - The endurance experiment has been executed, summarized, and incorporated into this final report.

Artifact note: - The persisted-message counts reported in the batch matrix were inferred from the saved metrics API responses because the copied experiment folders did not include **postgres-summary.txt** for those runs.